

Algorytmy Zaawansowane

Opracował Radosław Głasek na podstawie wykładów Michała Tuczyńskiego

Spis treści

1	Wykład 1	2
1.1	Notacja asymptotyczna — przypomnienie	2
1.2	Strategia typu <i>Dziel i Zwyciężaj</i> (<i>Divide and Conquer</i>)	3
2	Wykład 2	6
2.1	Algorytm Strasiery	6
2.2	Własność optymalnej podstruktury	7
2.3	Znajdywanie pary najbliższych punktów	7

Wykład 1

Notacja asymptotyczna — przypomnienie

$$f, g : \mathbb{N} \rightarrow \mathbb{R}_+$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} g(n) \leq c \cdot f(n)$$

$$g(n) = \Omega(f(n)) \Leftrightarrow \exists_{c \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c \cdot f(n) \leq g(n)$$

$$g(n) = \Theta(f(n)) \Leftrightarrow \exists_{c_1, c_2 \in \mathbb{R}_+} \exists_{n_0 \in \mathbb{N}_+} \forall_{n > n_0} c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)$$

$$g(n) = \mathcal{O}(f(n)) \Leftrightarrow f(n) = \Omega(g(n))$$

$$g(n) = \Theta(f(n)) \Leftrightarrow (g(n) = \mathcal{O}(f(n)) \wedge g(n) = \Omega(f(n)))$$

$$n^a = \mathcal{O}(n^b), \quad 0 < a \leq b$$

$$\log n = \mathcal{O}(n^\epsilon), \quad \epsilon > 0$$

$$n^a = \mathcal{O}(b^n), \quad 1 < a \leq b$$

$$\log_a n = \Theta(\log_b n), \quad a, b > 1$$

$$n^a = \mathcal{O}(b^n), \quad 0 < a, \quad b > 1$$

$$T(n) = 5n^3 + 2n^2 + 100n \log n \underset{n \rightarrow \infty}{\approx} 5n^3 = \Theta(n^3)$$

Procedura rekurencyjna

$$n! = \begin{cases} 1 & n = 0 \\ n \cdot (n-1)! & n \geq 1 \end{cases}$$

$$\text{Oczywiste jest, że } n! = \prod_{i=1}^n i$$

Ciąg Fibonacciego F_n - ciąg Fibonacciego.

$$F_n = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F_{n-1} + F_{n-2} & n \geq 2 \end{cases}$$

Algorytm 1: Rekurencyjna konstrukcja ciągu Fibonacciego

```

function FIBB( $n$ )
1:   if  $n = 0$  then return 0
2:   if  $n = 1$  then return 1
3:   return FIBB( $n - 1$ ) + FIBB( $n - 2$ )

```

Wywołanie rekurencyjne dla (typowo łatwiejszych i mniejszych) instancji tego samego problemu.

Strategia typu *Dziel i Zwyciężaj* (*Divide and Conquer*)

Algorytm typu *dziel i zwyciężaj* składa się z trzech części

1. *Dziel* – problem jest dzielony na mniejsze podproblemy
2. *Zwyciężaj* – podproblemy są rozwiązywane rekurencyjnie
3. *Połącz* – rozwiązanie całego problemu jest konstruowane z rozwiązań podproblemów

Zakładamy, że (zazwyczaj) w fazie *dziel* problem jest dzielony na co najmniej dwa podproblemy i podproblemy są rozłączne.

Przykład 1.1. Mnożenie liczb n -cyfrowych x, y – liczby naturalne n -cyfrowe.

Aby policzyć $x \cdot y$ można użyć mnożenia pisemnego (jak na kartce). Tym sposobem musimy wykonać $\mathcal{O}(n^2)$ mnożeń cyfr. Spróbujemy znaleźć lepszą metodę. Dzielimy x i y na „dwie połowy”. Zakładamy, dla uproszczenia, że n jest parzyste.

$$x = x_1 x_2 \cdots x_n = \underbrace{x_1 x_2 \cdots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \cdots x_n}_{x_R} = 10^{n/2} x_L + x_R$$

podobnie dla y :

$$y = \cdots = 10^{n/2} y_L + y_R$$

Wymnażamy:

$$x \cdot y = (10^{n/2} x_L + x_R) \cdot (10^{n/2} y_L + y_R) = 10^n x_L y_L + 10^{n/2} (x_L y_R + x_R y_L) + x_R y_R$$

Złożoność czasowa algorytmu – ozn. $T(n)$.

- Podział x, y na x_L, x_R, y_L, y_R zajmie $\mathcal{O}(n)$ czasu.
- Dodanie liczb n -cyfrowych zajmie $\mathcal{O}(n)$ czasu.
- Mnożenie przez 10^n jest równoważne dopisaniu n zer więc też $\mathcal{O}(n)$.

$$T(n) = 4 \cdot T(n/2) + \Theta(n)$$

Można lepiej, bo możemy zauważyć, że $x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$, więc potrzebujemy policzyć tylko 3 iloczyny: $x_L y_L, x_R y_R, (x_L - x_R) \cdot (y_R - y_L)$. To daje $T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$

Twierdzenie 1.2. *CLRS - Uniwersalne Twierdzenie o Rekurencji* Niech $a \geq 1, b > 1$ będą stałymi, $f(n)$ funkcją i $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & f(n) = \mathcal{O}(n^{\log_b a - \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & f(n) = \Omega(n^{\log_b a + \epsilon}) \text{ dla pewnego } \epsilon > 0 \\ & a \cdot f(n/b) \leq c \cdot f(n) \text{ dla pewnej stałej } c < 1 \text{ dla dużych } n \end{cases}$$

Lemat 1.3. Niech $a \geq 1, b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją zdefiniowaną dla potęg b . Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta(n^{\log_b n}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b a \cdot \log_b n} = (b^{\log_b a})^{\log_b n} = n^{\log_b a} \quad (1)$$

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a \left(f\left(\frac{n}{b}\right) + aT\left(\frac{n}{b^2}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2T\left(\frac{n}{b^2}\right) = f(n) + af\left(\frac{n}{b}\right) + a^2 \left(f\left(\frac{n}{b^2}\right) + aT\left(\frac{n}{b^3}\right) \right) \\ &= f(n) + af\left(\frac{n}{b}\right) + a^2f\left(\frac{n}{b^2}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n}}\right) + a^{\log_b n} T(1) \\ &\stackrel{1}{=} \dots + a^{\log_b n} T(1) = \dots + n^{\log_b a} T(1) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

□

Przeprowadzimy teraz dowód uniwersalnego twierdzenia o rekurencji (1.2). Ograniczymy się do przypadku, gdy n jest potęgą b . Pozostały przypadek jest bardziej techniczny.

Dowód. Rozważmy wszystkie przypadki:

1. $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$. Wtedy

$$f\left(\frac{n}{b^j}\right) = \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right)$$

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j \mathcal{O}\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) = \mathcal{O}\left(\underbrace{\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}}_{\blacktriangle}\right) \\ \blacktriangle &= \sum_{j=0}^{\log_b n - 1} a^j \frac{n^{\log_b a - \epsilon}}{b^{j(\log_b a - \epsilon)}} = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} a^j \frac{b^{j\epsilon}}{b^{j \log_b a}} \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\epsilon}{\underbrace{b^{\log_b a}}_{=a}}\right)^j = n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} b^{\epsilon j} \quad (\text{wzór na ciąg geometryczny}) \\ &= n^{\log_b a - \epsilon} \frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \frac{n^\epsilon - 1}{\underbrace{b^\epsilon - 1}_{\text{stała}}} \\ &= n^{\log_b a - \epsilon} \mathcal{O}(n^\epsilon - 1) = \mathcal{O}(n^{\log_b a - \epsilon} \cdot n^\epsilon - 1) = \mathcal{O}(n^{\log_b a}) \end{aligned}$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n - 1} a^j T\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a})$$

2. $f(n) = \Theta(n^{\log_b a})$. Wtedy:

$$\begin{aligned}
 f\left(\frac{n}{b^j}\right) &= \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n-1} a^j \Theta\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{b^{j \log_b a}}\right) = \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} a^j \frac{1}{\underbrace{b^{\log_b a^j}}_{=a^j}}\right) \\
 &= \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1\right) = \Theta(n^{\log_b a} \log_b n) \\
 \text{Z lematu: } T(n) &= \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \mathcal{O}(n^{\log_b a} \log n) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a} \log n)
 \end{aligned}$$

3. $f(n) = \Omega(n^{\log_b a + \epsilon})$ oraz $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ dla jakiegoś $c < 1$ i odpowiednio dużych n .

Z założenia $f\left(\frac{n}{b}\right) \leq \frac{c}{a} f(n)$, więc $f\left(\frac{n}{b^j}\right) \leq \left(\frac{c}{a}\right)^j f(n) = \frac{c^j}{a^j} f(n)$.

$$\begin{aligned}
 \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) &\leq \sum_{j=0}^{\log_b n-1} a^j \frac{c^j}{a^j} f(n) + \mathcal{O}(1) \leq f(n) \sum_{j=0}^{\log_b n-1} c^j + \mathcal{O}(1) \\
 &\leq f(n) \sum_{j=0}^{\infty} c^j + \mathcal{O}(1) \leq f(n) \frac{1}{1-c} + \mathcal{O}(1) = \mathcal{O}(f(n))
 \end{aligned}$$

$$\text{Z drugiej strony: } \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) = a^0 f\left(\frac{n}{b^0}\right) + \underbrace{\sum_{j=1}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right)}_{\geq 0} = \Omega(f(n))$$

$$\text{Z lematu: } T(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) + \Theta(n^{\log_b a}) = \Theta(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

□

Wniosek 1.4. Niech $a \geq 1$, $b > 1$ będą stałymi, a $f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$. Niech $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$ (gdzie $\frac{n}{b}$ znaczy $\left\lfloor \frac{n}{b} \right\rfloor$ lub $\left\lceil \frac{n}{b} \right\rceil$). Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & a > b^k \Leftrightarrow \log_b a > k \\ \Theta(n^{\log_b a} \log n) & a = b^k \Leftrightarrow \log_b a = k \\ \Theta(n^k) & a < b^k \Leftrightarrow \log_b a < k \end{cases}$$

Wróćmy teraz do naszego problemu mnożenia liczb n -cyfrowych. Drugi algorytm ma złożoność:

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 4, b = 2, k = 1$$

$$4 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Trzeci algorytm ma złożoność:

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \quad a = 3, b = 2, k = 1$$

$$3 > 2^1 \Rightarrow T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.59})$$

Wykład 2

Przykład 2.1. Mnożenie macierzy

A, B - macierze $n \times n$

$$A \cdot B = C = ?$$

Najpopularniejszy sposób mnożenia dwóch macierzy $n \times n$ działa w czasie $\mathcal{O}(n^3)$. Musimy obliczyć n^2 wartości i obliczenie każdej zajmuje liniowy czas.

$$B = \begin{bmatrix} \vdots \end{bmatrix} \quad A = \begin{bmatrix} \dots \end{bmatrix} \begin{bmatrix} \times \end{bmatrix} = C$$

Algorytm Strassera

Zakładamy, że n jest potęgą 2 (dla przejrzystości). Dzielimy A i B na 4 macierze $\frac{n}{2} \times \frac{n}{2}$:

$$A = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right], \quad B = \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right]$$

$$A \cdot B = \left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \cdot \left[\begin{array}{c|c} B_{11} & B_{12} \\ \hline B_{21} & B_{22} \end{array} \right] = \left[\begin{array}{c|c} C_{11} & C_{12} \\ \hline C_{21} & C_{22} \end{array} \right]$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

Zamieniliśmy 1 mnożenie macierzy $n \times n$ na 8 mnożeń macierzy $\frac{n}{2} \times \frac{n}{2}$ + podział i dodawanie ($\mathcal{O}(n^2)$). Stąd mamy zależność rekurencyjną:

$$T(n) = 8 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2) \quad a = 8, b = 2, k = 2$$

$$8 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 8}) = \Theta(n^3)$$

Strassen zauważył, że jeśli obliczymy

$$M_1 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

$$M_2 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$M_3 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

$$M_4 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_5 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_6 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_7 = (A_{21} + A_{22}) \cdot B_{11}$$

$$\text{to } C_{11} = M_1 + M_2 - M_4 + M_6$$

$$C_{12} = M_4 + M_5$$

$$C_{21} = M_6 + M_7$$

$$C_{22} = M_2 - M_3 + M_5 - M_7$$

Zatem

$$T(n) = 7 \cdot T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$$a = 7, b = 2, k = 2$$

$$7 > 2^2 \Rightarrow T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$$

Własność optymalnej podstruktury

Optymalne rozwiązanie problemu jest funkcją optymalnych rozwiązań podproblemów (optymalne rozwiązanie można skonstruować z optymalnych rozwiązań podproblemów). Żeby metoda *dziel i zwyciężaj* działała dobrze muszą być spełnione dwa warunki (przez problem i algorytm):

1. własność optymalnej podstruktury
2. podproblemy są rozłączne

Znajdywanie pary najbliższych punktów

Q - zbiór $n \geq 2$ punktów na płaszczyźnie

$$p_1 = (x_1, y_1) \quad p_2 = (x_2, y_2) \quad d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Problem: znaleźć parę punktów w najmniejszej odległości. Algorytm siłowy (*brute force*), który sprawdza wszystkie pary punktów działa w czasie $\Theta(n^2)$, bo mamy $\binom{n}{2} = \frac{n(n-1)}{2} = \Theta(n^2)$ par punktów. Pokażemy algorytm typu *dziel i zwyciężaj*, który działa w czasie $\mathcal{O}(n \log n)$.

W każdym wywołaniu rekurencyjnym wejście składa się z 3 rzeczy: zbioru $P \subset Q$ i dwóch tablic X i Y które zawierają wszystkie punkty z P . W tablicy X punkty są posortowane w porządku niemalejącym względem pierwszej współrzędnej, a w tablicy Y po drugiej współrzędnej.

Opis algorytmu:

jeśli $|P| \leq 3$ sprawdzamy wszystkie pary

jeśli $|P| > 3$

Dziel: Dzielimy punkty P pionową prostą l na dwa podzbiory P_L i P_R na lewo i na prawo od l tak, że $|P_L| = \left\lceil \frac{|P|}{2} \right\rceil$, $|P_R| = \left\lfloor \frac{|P|}{2} \right\rfloor$. Dzielimy tablicę X na X_L i X_R zawierające punkty odpowiednio P_L i P_R posortowane w porządku niemalejącym względem pierwszej współrzędnej. Podobnie dzielimy Y na Y_L i Y_R zawierające odpowiednio punkty z P_L i P_R posortowane w porządku niemalejącym po drugiej współrzędnej.

Zwyciężaj: Wywołujemy rekurencyjnie algorytm dla wejścia P_L, X_L, Y_L i P_R, X_R, Y_R . Algorytm znajduje pary najbliższych punktów w P_L i P_R . Niech δ_L i δ_R będą najmniejszymi odległościami znalezionymi przez algorytm dla odpowiednio P_L i P_R . Niech δ będzie mniejszą z nich: $\delta = \min(\delta_L, \delta_R)$.

Połącz: Para najbliższych punktów z P składa się z dwóch punktów z P_L lub z dwóch punktów z P_R lub jednego punktu z P_L i jednego z P_R .

Musimy sprawdzić czy istnieje para 3-go typu w odległości mniejszej niż δ . Jeśli taka para istnieje, wtedy jej punkty są w pionowym pasie o szerokości 2δ wokół l .

W celu sprawdzenia czy taka para istnieje wykonujemy następujące czynności:

1. Tworzymy tablicę Y' otrzymaną z Y poprzez usunięcie punktów spoza pasa. Y' jest posortowany w porządku niemalejącym względem drugiej współrzędnej.

2. Dla każdego punktu p z Y' sprawdzamy odległość między p , a p' dla 7 kolejnych punktów p' z Y' . Algorytm oblicza najmniejszą odległość δ' i zapamiętuje ją i odpowiednią parę punktów.
3. Jeśli $\delta' < \delta$ to pionowy pas zawiera parę punktów w mniejszej odległości niż pary znalezione w wywołaniach rekurencyjnych. Algorytm zwraca δ' i odpowiednią parę punktów. Wpp. zwraca δ i odpowiednią parę punktów.

Dowód poprawności. Jedyną rzeczą, która nie jest oczywista jest fakt, że sprawdzamy tylko 7 następnych punktów po p w Y' w fazie łącz. Przypuśćmy, że p, q są parą punktów w najmniejszej odległości w P i $\delta(p, q) = \delta'$. Niech $p = (x_p, y_p)$, $q = (x_q, y_q)$. Bez straty ogólności załóżmy $y_p \leq y_q$. Pokażemy, że algorytm znajdzie tę parę punktów. Przypuśćmy, że nie. Wtedy istnieje co najmniej 7 punktów pomiędzy p i q w Y' . Niech $r = (x_r, y_r)$ będzie jednym z nich. Skoro $d(p, q) < \delta$ to $y_q - y_p = |y_q - y_p| \leq \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2} = \delta(p, q) \leq \delta$. Ponadto y_r musi być między y_p i y_q : $y_p \leq y_r \leq y_q$, więc $y_r - y_p \leq y_q - y_p < \delta$ zatem co najmniej 9 punktów z P w prostokącie $2\delta \times \delta$ ograniczonym poziomymi liniami $y = y_p$, $y = y_p + \delta$.

Wtedy w lewym lub prawym kwadracie $\delta \times \delta$ jest przynajmniej 5 punktów z P . Przypuśćmy, że to lewy kwadrat. Zatem istnieje kwadrat $\delta \times \delta$ z 5 punktami z P w P_L . Ale wtedy istnieją dwa punkty w P_L w odległości mniejszej niż δ , bo nie da się umieścić w kwadracie $\delta \times \delta$ 5 punktów dla których odległość między dowolnymi dwoma jest mniejsza niż δ . Sprzeczność (bo najmniejsza odległość w P_L to $\delta_L \geq \delta$).

Zatem wystarczy sprawdzić tylko 7 następnych punktów po p w Y' . □

Analiza złożoności czasowej Na początku musimy posortować tablicę X i Y . To zajmuje $\mathcal{O}(n \log n)$. Jeśli X i Y są już posortowane to podział X na posortowane X_L i X_R oraz Y analogicznie zajmie $\mathcal{O}(n)$.

Algorytm 2: ???

```

1: length( $Y_L$ )  $\leftarrow$  length( $Y_R$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $Y(i) \in P_L$  then
4:     length( $Y_L$ )  $\leftarrow$  length( $Y_L$ ) + 1
5:      $Y_L(\text{length}(Y_L)) \leftarrow Y(i)$ 
6:   else
7:     length( $Y_R$ )  $\leftarrow$  length( $Y_R$ ) + 1
8:      $Y_R(\text{length}(Y_R)) \leftarrow Y(i)$ 

```

Mamy dwa wywołania rekurencyjne dla $n/2$ punktów i fazę łącz. Tablicę Y' można otrzymać z Y w czasie $\mathcal{O}(n)$ tak, że Y' jest posortowana. Niech $x = x_L$ będzie równanie prostej l .

Algorytm 3: ???

```

1: length( $Y'$ )  $\leftarrow$  0
2: for  $i \leftarrow 1$  to length( $Y$ ) do
3:   if  $x_L - \delta \leq Y(i).x \leq x_L + \delta$  then
4:     length( $Y'$ )  $\leftarrow$  length( $Y'$ ) + 1
5:      $Y'(\text{length}(Y')) \leftarrow Y(i)$ 

```

Skoro dla każdego z $\mathcal{O}(n)$ punktów z Y' sprawdzamy odległość tylko co najwyżej 7 punktów, to najmniejsza odległość δ' punktów z Y' znajdziemy w czasie $\mathcal{O}(n)$.

Oznaczmy złożoność algorytmu bez wstępnego sortowania przez $T(n)$. Mamy:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \ b = 2, \ k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n \log n)$$